Project Proposal, Requirements, Stakeholders - Phase 1

Team Name : *Vibe Coders* ***Date:*** *1/15/2026*

Team Members: *Wyatt Ladner, Jason Sonith, Ryan Tucker, Ralph Mouawad, Jonathan Gammill*

Project Name: *Multi-Agent AI System for Automated Medical Device Data Harvesting and Regulatory Data Validation*

Project document storage location (e.g. GoogleDocs): *Google Drive*

Project coding repository website link (e.g. github—): *Github*

Project Client: *Doug Greene, Fivos, doug.greene@fivoshealth.com*

Brief Project Description: *The FDA has this database called GUDID that's supposed to be the main source of truth for medical device info, but a lot of times it doesn't match what manufacturers actually have on their websites. Things like wrong dimensions or outdated brand names can cause problems with patient records and ordering equipment. Currently, entries must be manually checked, which is time consuming and prone to mistakes.. Our project for Fivos is building an AI system to automate this whole process.*

Describe the problem your completed project work will solve: *We're making two AI agents: Agent A (the Harvester) scrapes manufacturer websites and pulls device specs into a data lake, and Agent B (the Validator) compares that against GUDID and flags anything that doesn't match. Then there's a web dashboard where humans can review what the AI flagged and give feedback so it gets better over time. The dashboard also lets reviewers approve or reject each flagged item, For the tech stack we're planning on using Python, open source AI like Ollama, PostgreSQL or SQLite for the database, and GitHub for version control.*

Identify the stakeholders and how they will benefit from and be impacted by your project: *Our primary stakeholder is Doug Greene, the Project Sponsor from Fivos, who is based in Mobile, AL. He'll benefit from this project because right now his team has to manually verify thousands of medical device entries against manufacturer websites, which is time consuming and people make mistakes. Our AI system will automate most of that work so his team can focus on just reviewing the flagged discrepancies instead of checking everything by hand. This saves Fivos time and money while also improving the accuracy of their data.*

Additional stakeholders include the Fivos team members working from various US locations and Cairo, Egypt, as well as the human experts and reviewers who will ultimately use the HITL dashboard to review AI-flagged exceptions and provide feedback that improves the system over time.

Describe the Functional Requirements of the Project:

The functional requirements for this project are prioritized based on both their importance to the overall system and their complexity. The idea is to tackle the most critical and complex stuff first while working on multiple things in parallel when possible.

1. **The Harvester Agent** - Role: Automated System - This is basically a web scraper on steroids. It goes to manufacturer websites, deals with all the annoying dynamic content like pages that load with JavaScript, and pulls out the technical specs for medical devices. All that data gets dumped into a central database. This is the top priority because nothing else works if we don't have the data.
2. **The Validator Agent** - Role: Automated System - This one takes the data we scraped and compares it against what's in the FDA's GUDID database. If something doesn't match up, like the dimensions are different or a company changed their name, it flags it. This is the whole point of the project so it's obviously super important.
3. **Review Dashboard** - Role: Human Reviewer - This is the website where the Fivos team actually looks at the flagged problems. They can see the FDA data next to the manufacturer data and decide if there's actually an issue or if the system made a mistake. Only people with reviewer access should be able to use this.
4. **Feedback System** - Role: Human Reviewer - When reviewers make decisions about flagged items, the system remembers those choices and uses them to get better at catching real problems. Reviewers use this through the dashboard so it's tied to their role.
5. **Data Storage** - Role: System Administrator - Someone has to set up and maintain the database where all the scraped data lives. That's an admin job. Reviewers just see the data through the dashboard, they're not messing with the actual database.
6. **GUDID Import** - Role: System Administrator - Admins handle getting the FDA data into the system and keeping it updated. Reviewers don't need to worry about this backend stuff.
7. **Login and Permissions** - Role: System Administrator - Admins create accounts and decide who's a reviewer versus who's an admin. This keeps people from accessing stuff they shouldn't.
8. **Reports and Stats** - Role: Human Reviewer, System Administrator - Both roles can look at how the system is performing, like how many discrepancies are getting flagged and how accurate it is. Admins might get more detailed technical info that reviewers don't really need.

Describe the Non-Functional Requirements of the Project:

1. **Performance:** The system should finish a typical harvest/validate run in a reasonable amount of time so reviewers aren't waiting forever. The dashboard should also load pages quickly and not lag when switching between records. If a manufacturer site is slow, the harvester should use timeouts and retries instead of hanging the whole run.
2. **Reliability & Fault Tolerance:** The system needs to handle failures without crashing. Manufacturer websites might be down, block requests, or change layouts, so the harvester should log errors, retry a few times, and continue to the next device/site when possible. The validator should also handle missing GUDID records and messy data (mark them as unmatched for example).
3. **Accuracy & Data Quality:** The goal is to minimize false flags. Since website labels and GUDID fields won't always match, the system must normalize values (units, formatting, naming) before comparing. Discrepancies should include a confidence score and clear explanation so reviewers can trust what the system is reporting.
4. **Traceability & Auditability:** Every harvested value should be traceable to where it came from (source URL, timestamp etc.). Reviewer actions must be logged (who approved/rejected, when, and why). This is important so the process stays consistent and the team can explain how a final value was chosen.
5. **Security & Access Control:** Only authorized users should access the dashboard. Reviewer vs admin permissions should be enforced. Any API keys, credentials, or tokens used for scraping or data access must be stored securely and not hardcoded in the repo.
6. **Maintainability & Extensibility:** The system should be modular so we can add new manufacturers without rewriting everything. Code should be organized and documented enough that another student/team could continue the project later.
7. **Usability (HITL efficiency):** Usability (HITL efficiency): The dashboard needs to be easy and quick for reviewers to use. It should clearly show the FDA (GUDID) info next to the manufacturer info, with simple buttons to approve, reject, or edit a flagged item. When a reviewer makes a decision, the system should ask for a reason code so we can track why it happened. It should also have basic filters and sorting so reviewers can focus on the most important discrepancies first instead of scrolling through everything.
8. **Cost / Tooling Constraints:** The solution must use open-source or free tools. The design should avoid expensive paid services and be able to run locally on a dev machine or basic server.

Technologies/skills expected and required:

First, for the backend and AI agents, Python is definitely going to be the way to go. The document mentions they're hoping for Python across the board, which makes sense because

we'll need libraries like Selenium or Playwright to handle web scraping and navigating those manufacturer websites. For Agent A specifically, we'll probably need some experience with web automation and handling dynamic content, maybe some knowledge of APIs too if we can hit manufacturer endpoints directly. For databases, we'll need to understand SQL pretty well since they're talking about PostgreSQL or SQLite for structured data. But we also need to handle the data lake, which sounds like it's going to be a NoSQL database, probably MongoDB or something similar. So we're looking at experience with both relational and non-relational databases. On the frontend side, the document says we can use pretty much any popular stack. I'm thinking React or Next.js would be solid choices for building that HITL dashboard where people are reviewing the AI's work. We'd need someone comfortable with JavaScript and modern web design principles. We're also going to need someone who understands version control with Git and GitHub, since that's what they want for source control. Finally, we'll need at least basic networking knowledge to understand how data flows between all these components, and probably some DevOps knowledge to get everything deployed and running together properly.

Currently known constraints*.*

1) Technical Constraints: The system has to work with dynamic manufacturer websites, which is going to be tricky because these sites change constantly and use different structures. We can't just write one scraper and call it a day since we'll need to build something flexible that can adapt when websites update their layouts. We're also limited to open source or free tools, which is good for budget but might limit some functionality. Open source AI like Ollama is preferred, and we need to stick with free cloud options if we go that route. Same thing with databases since we can't just throw money at expensive enterprise solutions.

   GUDID field names and manufacturer website labels may not match, and they may use different units, formats, or levels of detail (for example “overall length” vs “shaft length”). Because of that, the system can’t assume one-to-one parameter equality and needs a normalization layer to translate website specs into a consistent internal schema before comparing to GUDID, otherwise it will generate lots of false discrepancies.

2) Organizational Constraints: Since our team is spread out across the U.S. and Cairo, most of our work and meetings will be online, so we have to stay organized with weekly check-ins, clear task assignments, and good documentation to avoid confusion. Also, the human reviewers won’t have time to look at every single flagged issue, so the system needs to help prioritize the most important discrepancies first. Lastly, we need to agree

on basic rules for what to do when GUDID and a manufacturer website disagree, and keep track of who approved a change and why so the process stays consistent.

3) Scope Constraints: We have three pretty specific milestones we need to hit: the design requirements document, the design specification with wireframes, and the final report. The project is ambitious since we're building two separate AI agents plus a whole dashboard interface, which is a lot to accomplish. Coordination between different parts of the system is going to be critical, and we need to make sure we actually deliver these milestones on time without the different components falling out of sync with each other.

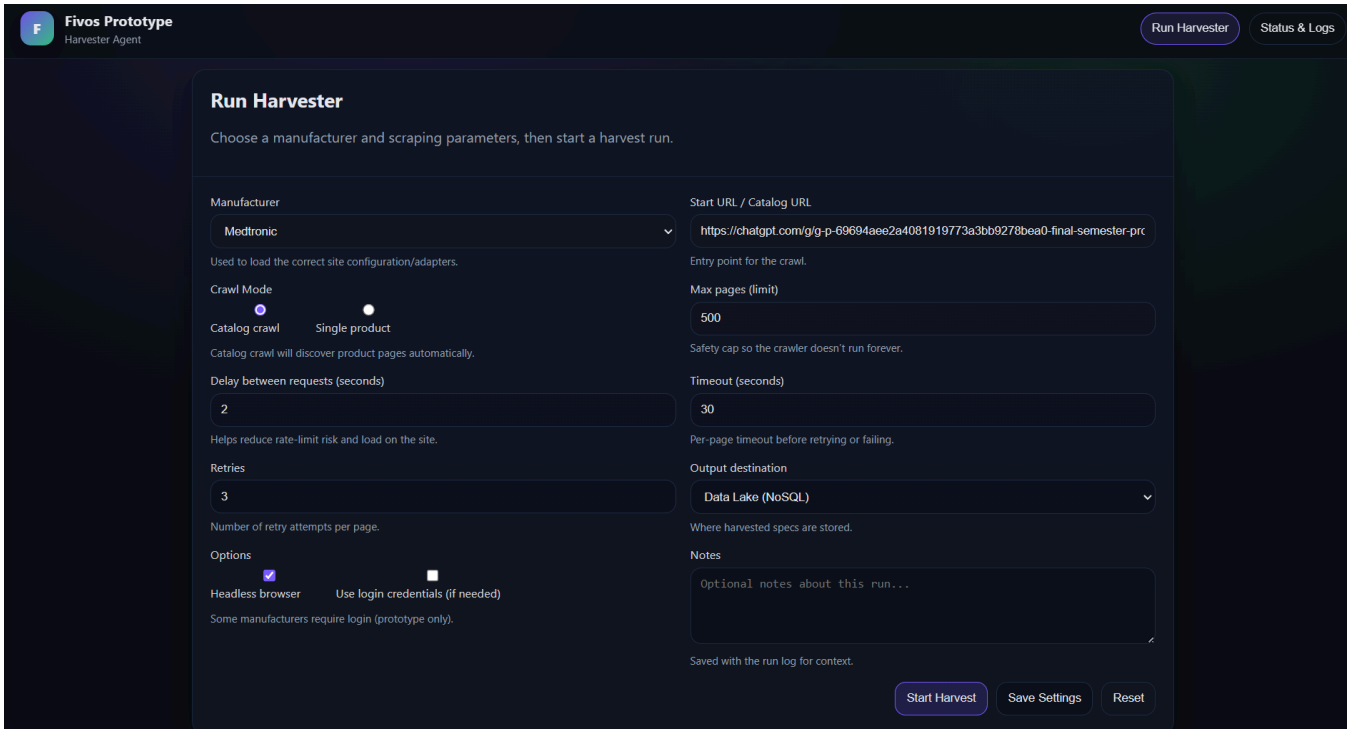
Complete a Use Case Narrative for each use case and for your three most important, highest priority use cases create System Input and System Output Screen Prototypes.

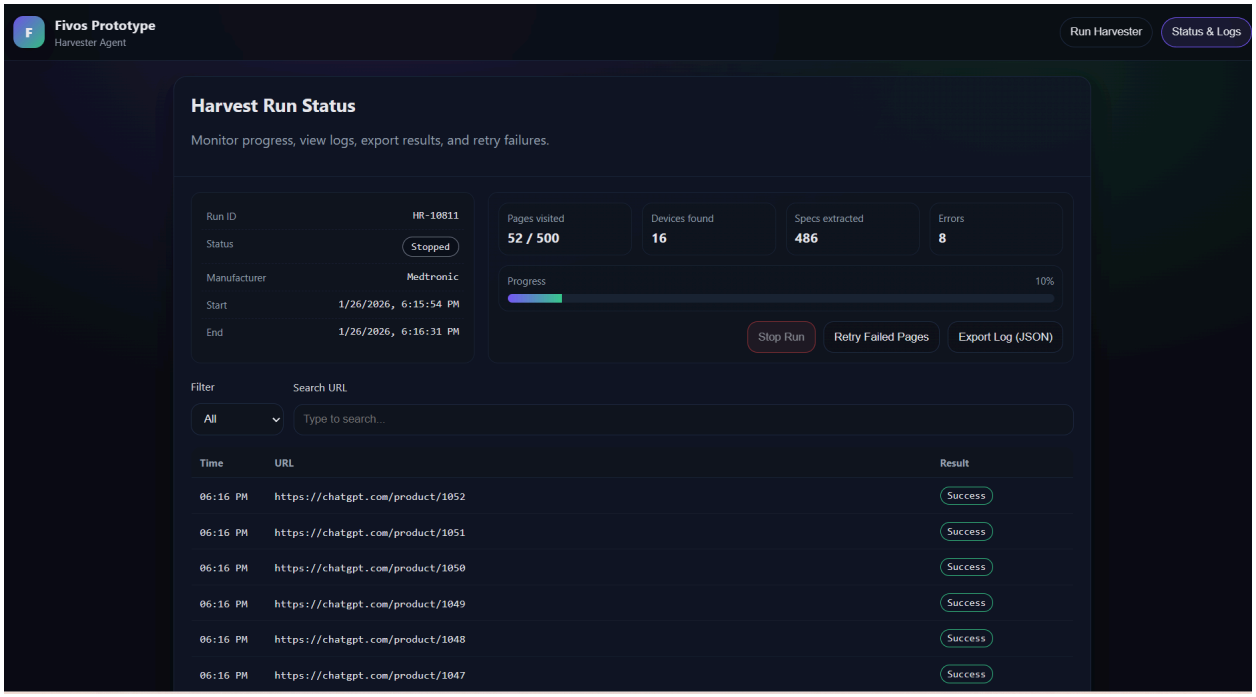
Use case:	**Harvest Device Data from Manufacturer**
Actors:	**Harvester Agent**
Preconditions:	The Harvester Agent must be set up and working, the target website must be reachable, and there must be an active internet connection. The database where we store the data also needs to be up and running.
Postconditions:	Device specifications have been extracted from the manufacturer website and stored in the data lake with a timestamp. A harvest log entry has been created documenting what was extracted and whether there were any errors. The data is now ready for the Validator Agent to compare against GUDID records.

Flow of Events	
Actor Action	**System Response**
The System Administrator or the Scheduler starts the harvester agent to extract data from the manufacturer's website	The system validates the manufacturer URL, establishes a connection to the website, and logs the start time of the harvest operation.
The Harvester Agent receives the manufacturer website URL and extraction parameters.	The system follows any redirects, waits for JavasScript to render content, and handles timeout with retries. If the page is loaded successfully, the system proceeds to extract data.

Alternative Flows of Events
Website URL is Invalid or Unreachable: If the system cannot establish a connection to the manufacturer website after 3 retry attempts with 5-second delays, the system logs the error with the HTTP status code, marks this harvest attempt as "Failed - Unreachable," and continues to the next URL in the queue if one exists. No data is harvested from this website.

System Input and Output Prototype for the use case “Harvest Device Data from Manufacturer”:




Fivos Prototype
Harvester Agent
Run Harvester
Status & Logs
Harvest Run Status
Monitor progress, view logs, export results, and retry failures.
Run ID HR-10811
Status Stopped
Manufacturer Medtronic
Start 1/26/2026, 6:15:54 PM
End 1/26/2026, 6:16:31 PM
Pages visited 52 / 500
Devices found 16
Specs extracted 486
Errors 8
Progress 10%
Stop Run
Retry Failed Pages
Export Log (JSON)
Filter
All
Search URL
Type to search...
Time URL Result
06:16 PM https://chatgpt.com/product/1052 Success
06:16 PM https://chatgpt.com/product/1051 Success
06:16 PM https://chatgpt.com/product/1050 Success
06:16 PM https://chatgpt.com/product/1049 Success
06:16 PM https://chatgpt.com/product/1048 Success
06:16 PM https://chatgpt.com/product/1047 Success

Use case:	**Validate data from GUDID and Manufacturer**
Actors:	**Validator Agent**
Preconditions:	Before the Validator Agent can run, the Harvestor Agent must have successfully collected data from manufacturer websites and stored it in the central database. The FDA GUDID database must be accessible through an API or data source, and the Validator must be active with a stable network connection to all required systems.
Postconditions:	After the validation process is complete, the manufacturer device data has been compared with corresponding UDID records. Any discrepancies are flagged. The validation results are stored in the database. The validation data is then ready for review, reporting, or further processing.

Flow of Events

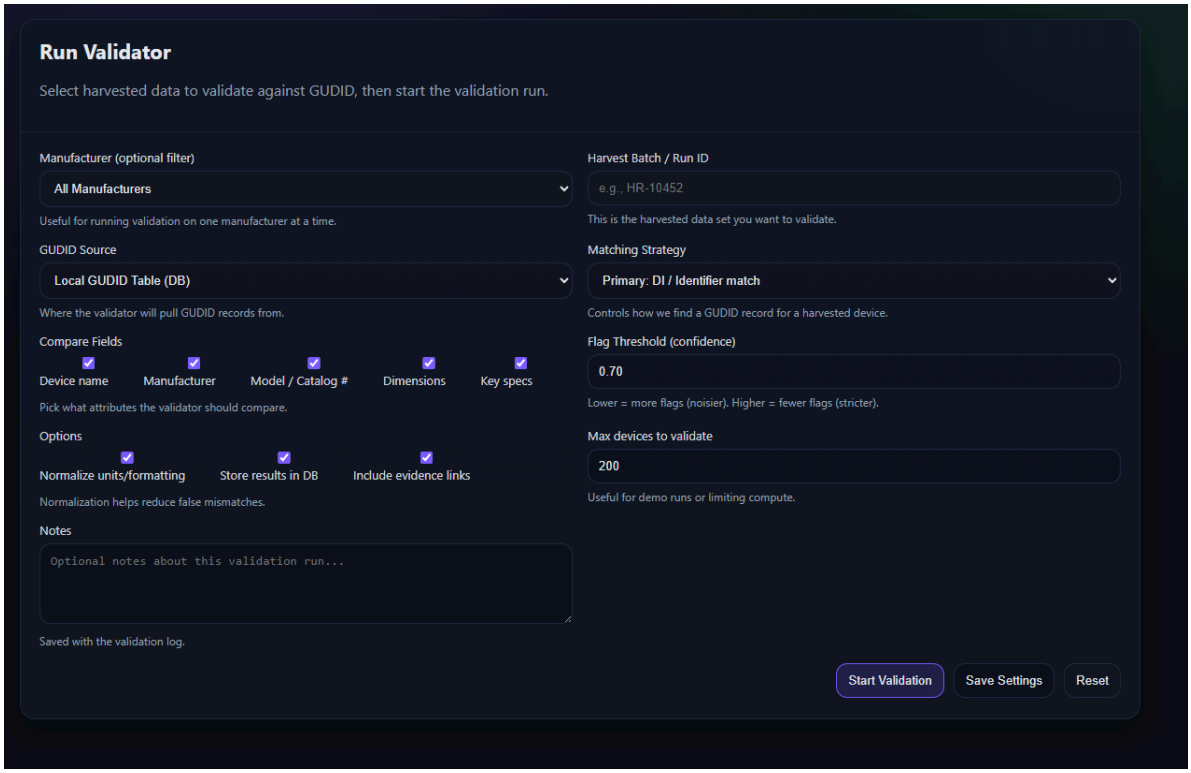
Actor Action	**System Response**
The System Administrator or Scheduler triggers the Validator Agent.	The system loads recently harvested device records and initializes the validation process.
The Validator Agent retrieves device data from the central database.	The system queries the FDA GUDID database for matching device records using identifiers such as device name, manufacturer, or GUDID ID.
The Validator Agent compares manufacturer data with GUDID data.	The system checks key attributes (e.g., device name, manufacturer, model, dimensions, specifications) and calculates match or mismatch results.
The Validator Agent evaluates discrepancies.	The system flags inconsistencies, categorizes them by severity, and assigns a validation status (Matched, Partial Match, or Failed).
The Validator Agent stores results.	The system saves validation outcomes and logs details of mismatches for auditing and review.

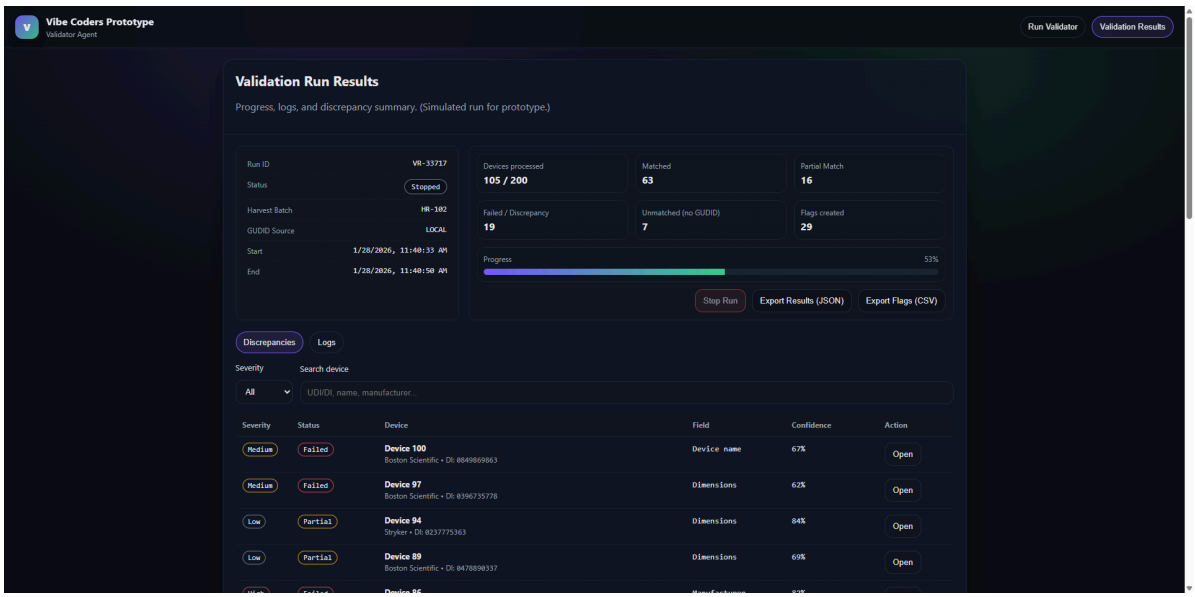
Alternative Flows of Events

GUDID record cannot be found: If a corresponding GUDID record cannot be found for a device, the system logs the issue, marks the record as unmatched, and continues validating the next device. If data fields cannot be directly compared due to formatting differences,

such as unit variations or naming conventions, the system attempts to normalize the data; if normalization fails, the record is marked as a partial match.

System Input and Output Prototype for the use case “Validate data from GUDID and Manufacturer” :





Use case:	**Review Flagged Discrepancies in Dashboard**
Actors:	**Human Reviewer**
Preconditions:	The reviewer must have a valid reviewer account and be logged into the dashboard. The Validator Agent must have already run and stored validation results in the database. There must be at least one flagged discrepancy available to review.
Postconditions:	The reviewer has reviewed one or more flagged discrepancies and marked them as approved, rejected, or corrected. The reviewer's decision (including a reason code and optional notes) is saved in the system and logged for auditing. Any resolved items are removed from the "pending review" queue and the feedback is available for improving future runs.

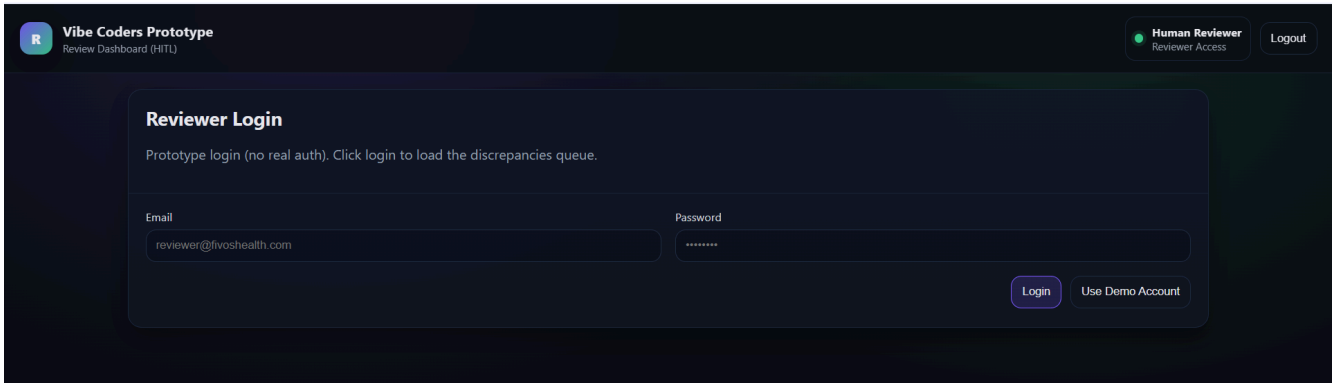
Flow of Events

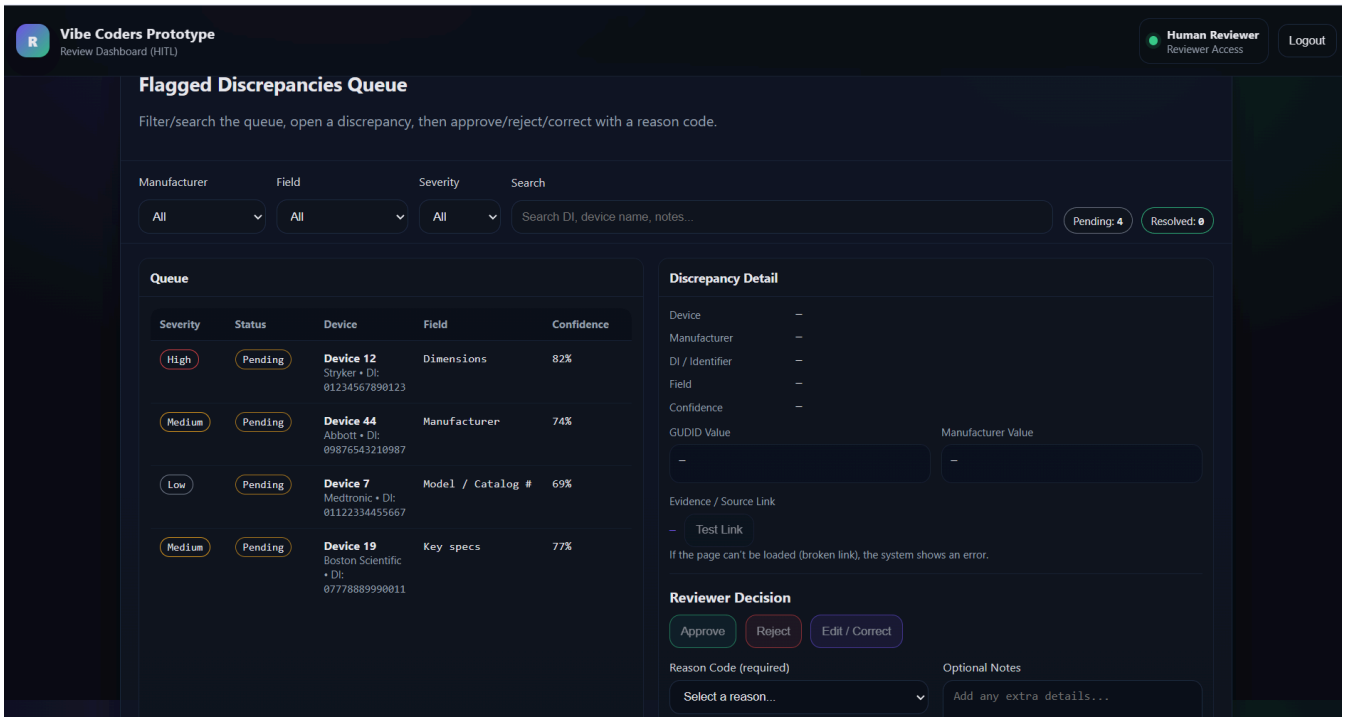
Actor Action	**System Response**
The Human Reviewer logs into the dashboard to review flagged issues.	The system verifies the reviewer's login/permissions and loads the discrepancies queue.
The Human Reviewer filters or searches the queue (ex: by manufacturer, field, or severity) and selects a discrepancy to open.	The system displays the discrepancy details, including the GUDID value next to the manufacturer value, plus the source link and confidence score.
The Human Reviewer reviews the information and chooses an action (Approve, Reject, or Edit/Correct).	The system prompts the reviewer to select a reason for action and allows optional notes.
The Human Reviewer submits their decision.	The system saves the decision, updates the discrepancy status (Resolved/Approved/Rejected/Corrected), logs the reviewer action with a timestamp, and updates the queue.

Alternative Flows of Events

Missing Evidence or Broken Source Link: If the manufacturer page can't be loaded or the source link is broken, the system shows an error message.

System Input and Output Prototype for the use case “Review Flagged Discrepancies in Dashboard” :





Use case:	**Import/Update GUDID Data**
Actors:	**System Administrator**
Preconditions:	The System Administrator must have admin access and be logged into the system (or have access to the admin tools). The database must be running and have enough storage. The GUDID data source (download file or API) must be available.
Postconditions:	GUDID records are successfully imported (or updated) in the database. The system logs the import (time, record count, success/failure). The Validator Agent can now use the updated GUDID data for comparisons.

Flow of Events

Actor Action	**System Response**
The System Administrator selects the "Import GUDID Data" option in the admin area.	The system shows import options (upload file or connect to data source) and the current GUDID version/date if available.
The System Administrator provides the GUDID data source (ex: uploads a file or enters the API/location info) and starts the import.	The system validates the input, starts the import process, and shows progress (ex: percent complete or record count).
The System Administrator confirms the import settings (ex: full replace vs incremental update).	The system writes the GUDID data into the database and normalizes fields into the system's schema.
The System Administrator finishes the import.	The system displays a success message with summary stats (records added/updated, errors if any) and saves an import log entry.

Alternative Flows of Events

Invalid File / Wrong Format: If the uploaded file is missing required fields or is in the wrong format, the system stops the import, shows an error message, and logs the failure.

Data Source Unreachable: If the system cannot access the GUDID source (network/API issue), the system logs the error and marks the import as failed without changing existing GUDID data.

Use case:	**Manage Login and Permissions**
Actors:	**System Administrator**
Preconditions:	System Administrator is logged into the system with their valid administrator credentials, the system is operational and accessible when administrator credentials are applied. The administrator has proper authorization to manage user accounts.
Postconditions:	A new user account is created successfully, then an appropriate role is assigned to the user. Users can access the features and data that is only permitted by their designed roles. Access restrictions are enforced through the system.

Flow of Events

Actor Action	**System Response**
System Administrator navigates to the User Management Section.	The system displays a current list of users and their assigned roles.
Administrator selects a way to create a new account.	The system then presents a user creation prompt that contains, “Username, Email Address, Temporary Password, and Role Selection (Reviewer or Administrator)”.
Administrator enters the required user information.	Administrator then selects the appropriate roles from the dropdown menu. Reviewer: Limited access to review and comment on content Administrator: Full system access including user management.
Administrator creates an account, “Create Account”.	System validates the entered informationSystem creates the user account with the assigned role. The system sends a welcome email to the new user with login credentials. The system displays a confirmation message: "User account created successfully". System updates the user list to include the new user.

Alternative Flows of Events

3a. Administrator wants to modify an existing user's role:

- 3a1: Administrator selects an existing user from the list
- 3a2: Administrator clicks "Edit User"
- 3a3: System displays the user's current information and role
- 3a4: Administrator changes the role assignment
- 3a5: Administrator clicks "Save Changes"
- 3a6: System updates the user's permissions based on the new role
- 3a7: System logs the role change for audit purposes
- 3a8: Use case continues at step 11

8a. Validation fails - Username being created already exits:

- 8a1: System displays error: "Username already exists. Please choose a different username"
- 8a2: Use case returns to step 5

8b. Validation fails - Invalid email format:

- 8b1: System displays error: "Invalid email format. Please enter a valid email address"
- 8b2: Use case returns to step 5

8c. Validation fails - Required fields are empty:

- 8c1: System highlights the empty required fields
- 8c2: System displays error: "Please fill in all required fields"
- 8c3: Use case returns to step 5

3b. Administrator wants to deactivate a user account:

- 3b1: Administrator selects a user from the list
- 3b2: Administrator clicks "Deactivate Account"
- 3b3: System prompts for confirmation: "Are you sure you want to deactivate this account?"
- 3b4: Administrator confirms the action
- 3b5: System deactivates the account and revokes all access permissions
- 3b6: System displays confirmation message
- 3b7: Use case ends

Use case:	**View System Performance Reports and Statistics**
Actors:	**Human Reviewer** **System Administrator**
Preconditions:	The actor is logged into the system with valid credentials. The system has collected performance data from device review. At least one complete review cycle has occurred (minimum 24 hours of data). AI discrepancy detection has processed device records
Postconditions:	The actor has viewed all relevant performance metrics for their specific role. Report data is logged for auditing purposes. The actor understands current system accuracy and efficiency. Exported reports are saved if they are generated.

Flow of Events

Actor Action	**System Response**
The Human Reviewer or System Administrator navigates to the *Reports & Statistics* section of the dashboard.	The system verifies the user's role and permissions. The system retrieves aggregated performance metrics from the database. The system displays role-appropriate statistics: Total discrepancies flagged, Percentage approved vs. rejected, Overall accuracy/confidence trends. Volume trends over time
The user optionally filters reports by date range, manufacturer, or discrepancy type.	The user optionally filters reports by date range, manufacturer, or discrepancy type.

Alternative Flows of Events

A1 – Administrator Views Advanced Metrics

- The **System Administrator** selects "Advanced Metrics."
- The system displays additional technical details, such as:
  - Validator confidence score distributions
  - False positive and false negative rates

- Processing time per validation run
- Model or rule version performance comparisons

- The administrator reviews the data to assess system tuning or retraining needs.

A2 – No Data Available

- The system detects that no validation data exists for the selected time range.
- The system displays a message indicating **no data is available** and suggests adjusting filters.

Exceptions

- **E1 – Unauthorized Access:**
  - If a reviewer attempts to access administrator-only metrics, the system denies access and displays a permission warning.
- **E2 – Data Retrieval Failure:**
  - If metrics cannot be retrieved, the system displays an error message and logs the failure.

Client sign-off: I _Doug Greene_____(print name) am the client for this senior design project. I have examined this document and discussed it with the team, particularly the functional requirements, and agree it is appropriate for my situation.

Signed __Doug Greene_____